EvilFlowers Search Service

API & Internal Class Reference

`evilflowers-search-service | FastAPI | Python 3.11+`

Overview

The search service is a FastAPI application that provides dual-index search over document chunks. It runs on port 8000 (mapped to 8001 on the host) and maintains two independent indexes: Elasticsearch for keyword search and Milvus for semantic (vector) search.

Documents are indexed by the text service via POST /index after PDF extraction and chunking. Search clients query either index independently via POST /search/elasticsearch or POST /search/semantic.

HTTP API

All endpoints accept and return JSON. Base URL inside Docker: http://search-service:8000. From the host: http://localhost:8001.

GET /health

Returns status and statistics for both Elasticsearch and Milvus. Use this to verify connectivity before running searches.

GET `/health`	
Description	
Request body	`(none)`
Response	`{ "status": "healthy", "milvus": { "total_entities": 0 }, "elasticsearch": { "connected": true, "total_chunks": 0 } }`

POST /index

Indexes a document into both Elasticsearch and Milvus. Existing entries for the document_id are deleted first, making this operation idempotent. The chunks field must contain the output of the text service processor.

POST `/index`

Description	
Request body	`{ "document_id": "doc-123", "chunks": { "chunks": [ { "text": "...", "metadata": { "chunk_index": 0, "source_page": 1, ... } } ] } }`
Response	`{ "document_id": "doc-123", "elasticsearch": { "indexed": 42, "errors": 0 }, "milvus": { "chunks_indexed": 42 } }`

DELETE /documents/{document_id}

Removes all chunks for a given document from both Elasticsearch and Milvus.

DELETE `/documents/{document_id}`	
Description	
Request body	`(none)`
Response	`{ "document_id": "doc-123", "milvus": { "chunks_deleted": 42 }, "elasticsearch": { "deleted": 42 } }`

POST /search/semantic

Semantic vector search via Milvus. The query is embedded using XLM-R and matched against stored embeddings using cosine similarity. Returns top_k results ranked by score (0-1, higher is better). Optionally filtered by document_id or page_num.

POST `/search/semantic`	
Description	
Request body	`{ "query": "neurónové siete", "top_k": 10, "document_id": "doc-123" (optional), "page_num": 5 (optional) }`
Response	`{ "search_type": "semantic", "query": "...", "results": [ { "id": 123, "score": 0.87, "document_id": "...", "source_page": 3, "text": "..." } ], "total_results": 10 }`

POST /search/elasticsearch

Keyword search via Elasticsearch. Uses multi_match best_fields with fuzziness=AUTO and operator=AND. Returns highlighted matches. Optionally filtered by document_id.

POST `/search/elasticsearch`	
Description	

Request body	`{ "query": "derivacia", "top_k": 10, "document_id": "doc-123" (optional) }`
Response	`{ "search_type": "elasticsearch", "query": "...", "results": [ { "id": "doc-123:7", "score": 14.9, "document_id": "...", "highlight": { "text": ["..."] } } ], "total_results": 10 }`

GET /documents/{document_id}/chunks

Returns the first N stored chunks for a document from Elasticsearch. Useful for inspecting what was indexed. Default limit is 10, configurable via query param ?limit=N.

GET `/documents/{document_id}/chunks`	
Description	
Request body	`(none)`
Response	`{ "document_id": "doc-123", "chunks": [ { "id": "doc-123:0", "source": { ... } } ] }`

Internal Classes

ElasticService

Async wrapper around the Elasticsearch client. Handles index lifecycle, bulk indexing, keyword search, and stats. All methods are async and safe to call concurrently.

Constructor

`__init__(host, index_name, username, password, api_key, verify_certs, ca_certs, request_timeout, search_fields)`	All parameters optional; defaults read from environment variables (ELASTICSEARCH_HOST, ELASTICSEARCH_INDEX, etc.).

Methods

`default_mapping() -> dict`	Returns the default index mapping with typed fields for document_id, chunk_id, text, title, page, metadata, and created_at.
`async check_connection() -> bool`	Pings Elasticsearch and returns True if reachable, False on any error.

`async ensure_index(mapping=None, settings=None) -> None`	Creates the index with the given or default mapping if it does not already exist; no-op if it exists.
`async index_chunk(document_id, chunk_id, text, title=None, page=None, metadata=None, refresh=False) -> dict`	Indexes a single chunk using a deterministic composite ID (document_id:chunk_id) that allows safe re-indexing.
`async index_document(document_id, chunks, refresh=False) -> dict`	Bulk-indexes all chunks for a document, skipping any entries missing chunk_index or text. Returns count of indexed and errored chunks.
`async delete_document(document_id, refresh=False) -> dict`	Deletes all chunks belonging to document_id using a term query. Returns count of deleted documents.
`async search_documents(query, document_id=None, size=10, from_=0) -> list[dict]`	Runs a fuzzy multi-field full-text search with optional document filter. Returns hits with score, source, and highlighted fragments.
`async get_chunk(document_id, chunk_id) -> dict | None`	Fetches a single chunk by its composite ID; returns None if not found.
`async stats_overview() -> dict`	Returns aggregate index stats: total chunks, unique document count, deleted doc count, and store size in bytes.
`async close() -> None`	Closes the underlying HTTP connection pool.

SemanticService

Orchestrates embedding generation and Milvus operations. Acts as the high-level interface for all semantic search functionality.

`__init__()`	Instantiates EmbeddingGenerator (loads XLM-R model) and MilvusClient (connects and loads collection).
`index_document(document_id, chunks) -> dict`	Generates embeddings for all chunk texts in batch, then inserts them into Milvus with

	metadata. Returns count of indexed chunks and embedding dimension.
`search(query, top_k=10, document_id=None, page_num=None) -> list[dict]`	Embeds the query and runs a COSINE similarity search in Milvus. Optional filters on document_id and/or page_num.
`delete_document(document_id) -> dict`	Deletes all Milvus embeddings for the given document_id. Returns count of deleted entries.

EmbeddingGenerator

Wraps the sentence-transformers SentenceTransformer model (stsb-xlm-r-multilingual, 768-dim). Runs on CPU by default; configurable via EMBEDDING_DEVICE env var.

`generate_embeddings(texts, normalize=True, show_progress=False) -> np.ndarray`	Encodes a list of strings in batches (BATCH_SIZE from config). Returns a 2D float32 array of shape (N, 768).
`generate_single_embedding(text, normalize=True) -> np.ndarray`	Convenience wrapper that encodes one string and returns a 1D array of shape (768,).

MilvusClient

Low-level Milvus wrapper. Manages the collection lifecycle, HNSW index creation, and vector insert/search/delete operations.

`__init__()`	Connects to Milvus at MILVUS_HOST:MILVUS_PORT, creates the collection and HNSW index if they do not exist, and loads the collection into memory.
`insert_embeddings(document_id, embeddings, metadata) -> list[int]`	Inserts a batch of embeddings and their associated metadata into the collection. Returns list of primary key IDs assigned by Milvus.
`search(query_embedding, top_k=10, document_id=None, page_num=None) -> list[dict]`	Runs an HNSW COSINE search with optional expression filters. Returns top_k hits with score, distance, and all output fields.

`delete_by_document_id(document_id) -> int`	Deletes all entities matching document_id and flushes the collection. Returns count of deleted entities.
`get_stats() -> dict`	Returns collection name, total entity count, embedding dimension, index type, and metric type.
`close() -> None`	Releases the collection from memory and disconnects from Milvus.

Request & Response Schemas

Defined in schemas.py using Pydantic BaseModel.

IndexRequest

`document_id: str`	Unique identifier for the document (e.g. acquisition UUID from EvilFlowersCatalog).
`chunks: dict`	Dict with key 'chunks' containing a list of chunk objects, each with 'text' and 'metadata' fields.

SemanticSearchRequest

`query: str`	Free-text search query; embedded at request time.
`top_k: int = 10`	Number of results to return.
`document_id: Optional[str] = None`	Restrict search to a single document.
`page_num: Optional[int] = None`	Restrict search to a specific page within a document.

ElasticsearchSearchRequest

`query: str`	Keyword query string; supports fuzziness across text fields.

`top_k: int = 10`	Number of results to return.

`document_id: Optional[str] = None`	Restrict search to a single document.